Data Wrangling & Exploration in Python

A Comprehensive Guide Using the GSS Subset Dataset

Victor Wandera Lumumba
Statistician | Data Analyst | ML Expert
Mediacrest Training College

June 22, 2026

Abstract

Data wrangling is the foundational step in any data science pipeline. It involves cleaning, structuring, and enriching raw data into a format suitable for analysis and modeling. These notes provide a comprehensive, step-by-step guide to data wrangling and exploration using Python's `pandas` library. We will explore real-world techniques for handling missing values, detecting outliers, filtering data, engineering features, and generating descriptive statistics using the General Social Survey (GSS) subset dataset.

Contents

1 Introduction to Data Wrangling

Data wrangling (or data munging) is the process of cleaning, transforming, and mapping data from one "raw" form into another format that is more appropriate for analytics. In the real world, data is rarely clean. It contains missing values, duplicates, outliers, and inconsistent formatting. Mastering data wrangling is crucial because the quality of your insights and machine learning models is directly bounded by the quality of your data.

2 1. Data Importation

The first step in any analysis is loading the data into memory. The Comma-Separated Values (CSV) format is the universal standard for tabular data. Pandas provides the `read_csv()` function, which is highly optimized and handles parsing, type inference, and header alignment automatically.

Key Concept

Why Proper Import Matters: Properly defining data types and handling missing value placeholders during import prevents downstream errors. For instance, if a numeric column contains a string like `"Not Available"`, Pandas might infer the entire column as an `object` (string) type, breaking mathematical operations later.

```
import pandas as pd
import numpy as np

# Load the dataset
df = pd.read_csv('GSSsubset1.csv')

# Verify successful import
print(f"Shape: {df.shape}")
print(f"Columns: {df.columns.tolist()}")
```

Key Parameters of `read_csv()`:

- `sep`: Specifies the delimiter (default is `','`).
- `header`: Row number to use as column names (default is `0`).
- `na_values`: Additional strings to recognize as NA/NaN (e.g., `['?', '-99']`).
- `dtype`: Explicitly define data types for specific columns to save memory.

3 2. Initial Data Inspection

Before cleaning or analyzing, you must understand the structure and distribution of your data. This trio of methods is your first diagnostic checkpoint.

3.1 2.1 The `.info()` Method

The `.info()` method provides a concise summary of the DataFrame, including the index dtype and columns, non-null values, and memory usage. It is the fastest way to identify missing data and verify data types.

```
df.info()
```

3.2 2.2 The `.describe()` Method

The `.describe()` method generates descriptive statistics that summarize the central tendency, dispersion, and shape of a dataset's distribution. By default, it only analyzes **numeric** columns.

```
# Drop 'id' as it's just an identifier, not a meaningful numeric variable
df.drop(columns=["id"]).describe()
```

3.3 2.3 Frequency Tables for Categorical Variables

For categorical data (like `sex`, `degree`, `marital`), `.describe()` is not useful. Instead, we use `.value_counts()`. We can build a custom function to generate a comprehensive frequency table including percentages and cumulative percentages.

```
def freq_table(df, column):
    # Absolute frequencies
    freq = df[column].value_counts(dropna=False)
    # Percentages (normalize=True converts counts to proportions)
    percent = df[column].value_counts(normalize=True, dropna=False) * 100
    # Cumulative percentages
    cum_percent = percent.cumsum()

    # Combine into a single DataFrame
    table = pd.DataFrame({
        'Frequency': freq,
        'Percent (%)': percent.round(2),
        'Cumulative (%)': cum_percent.round(2)
    })
    return table

# Usage
print(freq_table(df, 'degree'))
```

4 3. Data Cleaning: Missing Values

Missing data appears as `NaN`, `None`, or placeholders. It can severely skew statistical analyses and cause machine learning algorithms to crash.

Mathematical Foundation

Types of Missing Data:

- **MCAR (Missing Completely At Random):** The missingness is entirely random (e.g., a sensor randomly failing).
- **MAR (Missing At Random):** The missingness is related to observed data (e.g., younger people are less likely to report their income).
- **MNAR (Missing Not At Random):** The missingness is related to the unobserved data itself (e.g., very high earners refuse to disclose income).

4.1 3.1 Detection and Visualization

```
# Count missing values per column
missing_counts = df.isna().sum()

# Calculate percentage of missing data
missing_pct = (df.isna().mean() * 100).round(2)
print(missing_pct[missing_pct > 0])
```

Note: In practice, visualizing missingness using libraries like ***missingno*** *(heatmaps, bar charts) helps identify patterns, such as whether missing values in one column correlate with missing values in another.*

4.2 3.2 Handling Strategies

4.2.1 Strategy 1: Dropping Data

Use `dropna()` cautiously. If a column has 90% missing data, or a row has almost all missing values, dropping them is acceptable. However, dropping rows indiscriminately can drastically shrink your dataset and introduce bias.

```
# Drop rows with ANY missing value (often too aggressive)
df_drop_all = df.dropna()
```

4.2.2 Strategy 2: Standard Imputation

Imputation fills missing values with a statistical measure.

- **Numeric Data:** Use the **Median** for skewed data (like income or weight) because the mean is highly sensitive to outliers. Use the **Mean** only for normally distributed data.
- **Categorical Data:** Use the **Mode** (most frequent category).

```
# Automatic numeric imputation using Median
numeric_cols = df.select_dtypes(include='number').columns
df[numeric_cols] = df[numeric_cols].fillna(df[numeric_cols].median
    ())
```

```
4
5  # Categorical imputation using Mode
6  cat_cols = df.select_dtypes(include=['object']).columns
7  for col in cat_cols:
8      mode_val = df[col].mode()
9      if not mode_val.empty:
10         df[col] = df[col].fillna(mode_val[0])
```

4.2.3 Strategy 3: Machine Learning Approach

For robust pipelines, Scikit-Learn's `SimpleImputer` is preferred. It separates the imputation logic from the DataFrame manipulation.

```
1  from sklearn.impute import SimpleImputer
2
3  num_cols = df.select_dtypes(include='number').columns
4  imputer = SimpleImputer(strategy='median')
5
6  # fit() calculates the median; transform() applies it
7  df[num_cols] = imputer.fit_transform(df[num_cols])
```

5 4. Data Cleaning: Duplicates

Duplicates inflate counts, bias averages, and violate unique constraints (like primary keys).

```
1  # Count exact duplicate rows
2  dup_count = df.duplicated().sum()
3  print(f"Exact duplicate rows found: {dup_count}")
4
5  # Remove duplicates (keeps the first occurrence by default)
6  df_clean = df.drop_duplicates()
7
8  # Check uniqueness of primary key (e.g., 'id')
9  print(f"ID uniqueness: {df['id'].nunique()} unique / {len(df)}
       total records")
```

Pro Tip

If your primary key (`id`) is not unique, you may have survey re-submissions or data entry errors. Investigate these specific rows before blindly dropping them!

6 5. Data Cleaning: Outliers

Outliers are extreme values that deviate significantly from other observations. They can be caused by data entry errors (e.g., weight = 999 lbs) or represent genuine extreme cases.

Mathematical Foundation

The Interquartile Range (IQR) Method: The IQR is the middle 50% of the data.

$$\begin{aligned} Q1 &= 25^{\text{th}} \text{ percentile} \\ Q3 &= 75^{\text{th}} \text{ percentile} \\ IQR &= Q3 - Q1 \\ \text{Lower Bound} &= Q1 - 1.5 \times IQR \\ \text{Upper Bound} &= Q3 + 1.5 \times IQR \end{aligned}$$

Any value outside the bounds is considered an outlier.

6.1 5.1 Handling Outliers: Winsorizing (Capping)

Instead of deleting outlier rows (which loses data), we can "cap" or "winsorize" them by setting a maximum and minimum threshold.

```
numeric_cols = df.select_dtypes(include=['float64', 'int64']).
    columns

for col in numeric_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1

    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    # Cap values using np.where
    df[col] = np.where(df[col] < lower_bound, lower_bound, df[col
    ])
    df[col] = np.where(df[col] > upper_bound, upper_bound, df[col
    ])

print("Outliers have been handled (capped) using the IQR method.")
```

7 6. Filtering Rows

Filtering allows you to subset data based on logical conditions. This is done using **Boolean masking**.

Warning

When combining multiple conditions in Pandas, you **must** wrap each condition in parentheses `()` and use the bitwise operators `&` (AND), `|` (OR), and `~` (NOT). Do not use Python's standard `and`, `or`, `not`.

```
# Single condition
males = df[df['sex'] == 'MALE']

# Multiple AND conditions
subset = df[(df['sex'] == 'FEMALE') &
            (df['age'] > 35) &
            (df['marital'] == 'MARRIED')]

# OR condition
high_educ = df[(df['degree'] == 'BACHELOR') | (df['degree'] == '
   GRADUATE')]
```

7.1 6.1 Advanced Filtering

- `.query()`: Uses string expressions. Highly readable for complex filters.
- `.isin()`: Replaces long `|` chains when checking membership in a list.
- **`~` operator:** Negates a condition (NOT IN).

```
# Cleaner syntax with query()
df_filtered = df.query('age >= 25 and income > 50000')

# Filter using a list of values with isin()
degrees_of_interest = ['BACHELOR', 'GRADUATE', 'JUNIOR COLLEGE']
df_edu = df[df['degree'].isin(degrees_of_interest)]

# NOT IN with ~ operator
df_no_hs = df[~df['degree'].isin(['LT HIGH SCHOOL', 'HIGH SCHOOL'
   ])]
```

8 7. Selecting Columns

Extracting specific variables saves memory and focuses your analysis.

8.1 7.1 Standard Selection and `.loc` / `.iloc`

```
# Multiple columns selection (returns DataFrame)
cols = ['sex', 'age', 'income', 'degree']
df_subset = df[cols]

# Label-based selection with .loc (inclusive of end label)
df_labels = df.loc[:, 'age':'weight']

# Position-based selection with .iloc
df_positions = df.iloc[:, [0, 2, 3, 7]] # id, degree, income,
   weight
```

8.2 7.2 Automated Selection by Data Type

When preparing data for machine learning, you often need to separate numeric and categorical features.

```
# Filter columns by data type
numeric_cols = df.select_dtypes(include='number').columns.tolist()
categorical_cols = df.select_dtypes(include='object').columns.
    tolist()

# Drop unwanted columns
df_dropped = df.drop(columns=['id', 'height'])
```

9 8. Grouping By (Split-Apply-Combine)

The **Split-Apply-Combine** paradigm is essential for comparative analysis. You split the data into groups, apply a function (like mean or sum) to each group independently, and combine the results.

```
# Basic grouping: Mean income by education level
edu_income = df.groupby('degree')['income'].mean().sort_values(
    ascending=False)

# Group by multiple columns
grouped = df.groupby(['sex', 'marital'])['income'].mean()

# Unstack to create a pivot-table-like view
grouped.unstack()
```

9.1 8.1 Advanced Aggregations

The `.agg()` function allows you to apply different statistical functions to different columns simultaneously.

```
# Apply different functions to different columns
summary = df.groupby('degree').agg({
    'income': ['mean', 'median', 'std', 'count'],
    'age': 'mean',
    'hrswrk': 'sum'
})

# Flatten multi-level column names for easier downstream use
summary.columns = ['_'.join(col).strip() for col in summary.
    columns]
print(summary)
```

10 9. Mutating: Feature Engineering

Feature engineering is the process of creating new variables from existing ones. This can dramatically improve model performance and interpretability. In our case, we want to

convert the continuous `income` variable into a categorical `Income_Level` variable.

10.1 9.1 Using `np.where` (Vectorized If-Else)

`np.where` is extremely fast and ideal for binary splits.

```
median_income = df['income'].median()

# Syntax: np.where(condition, value_if_true, value_if_false)
df['Income_Level'] = np.where(
    df['income'] >= median_income,
    'High Income',
    'Low Income'
)
```

10.2 9.2 Using `pd.cut` for Binning

If you need ordered categorical bins with explicit edges, `pd.cut` is the standard tool. It automatically handles NaNs and preserves the order of categories.

```
# Define bins and labels
bins = [0, median_income, np.inf]
labels = ['Low Income', 'High Income']

# pd.cut handles edges cleanly
df['Income_Level_cut'] = pd.cut(
    df['income'], bins=bins, labels=labels, include_lowest=True
)

# Verify it is an ordered categorical variable
print(df['Income_Level_cut'].cat.ordered)
```

11 10. Descriptive Statistics

Descriptive statistics summarize the central tendency, dispersion, and shape of a dataset's distribution.

11.1 10.1 Shape Metrics

- **Skewness:** Measures asymmetry. A positive skew (right-skewed) means the tail is on the right (common in income data). A negative skew means the tail is on the left.
- **Kurtosis:** Measures the "tailedness" (outliers). High kurtosis indicates heavy tails and a sharp peak; low kurtosis indicates a flat peak with thin tails.

```
print(f"Income Skewness: {df['income'].skew():.3f}")
print(f"Income Kurtosis: {df['income'].kurtosis():.3f}")
```

11.2 10.2 Coefficient of Variation (CV)

The CV is a measure of relative variability. It is the ratio of the standard deviation to the mean, expressed as a percentage. It is useful for comparing the dispersion of two datasets with different units or vastly different means.

$$CV = \frac{\sigma}{\mu} \times 100 \tag{1}$$

```
for col in ['income', 'age', 'weight', 'hrswrk']:
    cv = df[col].std() / df[col].mean()
    print(f"{col} CV: {cv:.3f} ({cv*100:.1f}%)")
```

12 11. Correlation Analysis

Correlation analysis examines the strength and direction of linear relationships between two continuous numeric variables.

Mathematical Foundation

Pearson Correlation Coefficient (r):

$$r = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^{n}(x_i - \bar{x})^2 \sum_{i=1}^{n}(y_i - \bar{y})^2}} \tag{2}$$

The value of r always falls between **-1.0 and 1.0**.

- $r = 1.0$: Perfect positive linear correlation.
- $r = 0.0$: No linear correlation.
- $r = -1.0$: Perfect negative linear correlation.

```
# Select numeric columns for correlation
numeric_vars = ['age', 'height', 'weight', 'hrswrk', 'income']
corr_matrix = df[numeric_vars].corr()
print(corr_matrix.round(3))
```

Warning

Correlation $\neq$ Causation! Just because two variables move together does not mean one causes the other. Always consider context, domain knowledge, and potential confounding variables before drawing causal conclusions.

13 12. Exporting Cleaned Data

The final step in data wrangling is saving your processed data. This ensures **reproducibility** and allows you to share the clean dataset with others or feed it directly into a machine learning pipeline.

```python
# Create a final cleaned dataset
df_export = df.copy()

# Example: Add a derived variable (BMI)
# BMI = (weight in lbs * 0.453592) / (height in inches * 0.0254)^2
df_export['BMI'] = (df_export['weight'] * 0.453592) / \
                   ((df_export['height'] * 0.0254) ** 2)

# Select relevant columns for final export
columns_to_keep = ['id', 'sex', 'degree', 'income', 'marital',
                   'age', 'height', 'weight', 'hrswrk', '
   Income_Level', 'BMI']
df_export = df_export[columns_to_keep]

# Save to CSV (index=False prevents Pandas from writing row
   numbers)
output_file = 'GSSsubset_processed.csv'
df_export.to_csv(output_file, index=False)

print(f"Processed dataset saved to: {output_file}")
```

14 Conclusion

Data wrangling is an iterative process. You will often find yourself moving back and forth between inspection, cleaning, and feature engineering as you uncover the nuances of your dataset. By mastering Pandas' powerful tools for handling missing data, outliers, and complex aggregations, you lay a rock-solid foundation for any advanced statistical analysis or machine learning task.